ELG 4157 Design Simulate 2

1st Jonathan Florus
300178425
jflor030@uottawa.ca

***Abstract*—An Artificial Neural Network (ANN) implemented using gradient descent training with data being generated from the Enhanced Dung Beetle Optimization and Grid Search methods is developed for the speed control of a separately excited DC Motors (SEDM). The goal of this methodology is to minimize the ITAE criterion of the DC motor and compare training data methodologies versus conventional methods of obtaining PI constants to achieve this goal. Performance comparisons between different implementations of the PI controller are discussed as well as comparisons with non-PI-controlled systems and PI-controlled systems with prefiltering. Simulation results are from a combination of Python and MATLAB Simulink.**

I. Introduction

In this project, we are examining a PI controlled DC Motor with negative feedback. The Motor has a dynamic load adjustment that will be simulated and compared against other PI-controlled, non-PI controlled, and pre-filtered systems.

II. Topology

The use of speed control for DC motors is simpler and less costly than AC motors. As a result, there is a plethora of applications for separately excited DC motors in variable speed and position control uses. This is due to their independent control of field and armature currents, where current flows towards the armature winding directly while there is a DC supply to the commutator and brushes.

DC motor speed is influenced by armature current, applied voltage, field flux, and armature resistance. As a result of these variables, the control of speed is dependent on voltage, field flux control, and armature resistance.

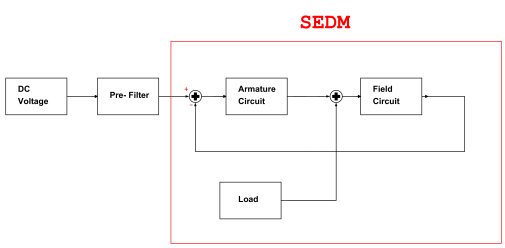

Fig. 1: Separately Excited DC Motor

The diagram above (Figure 1) illustrates the configuration of a separately excited DC motor.

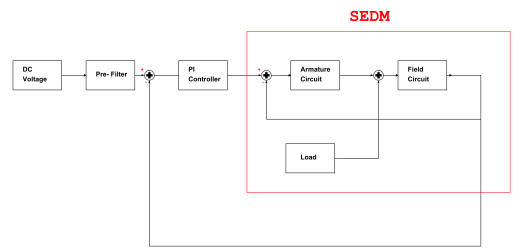

Fig. 2: Separately Excited DC Motor With PI Controller

Figure 2 shows the separately excited DC motor with a PI controller, highlighting the control system employed for optimal performance.

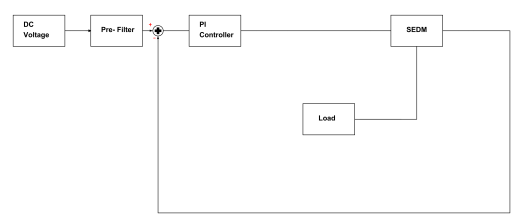

Fig. 3: Separately Excited DC Motor With PI Controller and Prefilter

In Figure 3, a simplified circuit of the DC motor with the PI controller is depicted, illustrating the system's components.

Closed-loop SEDM system (negative feedback)	
Metric	**Value**
ITAE	0.499061
Settling Time (s)	2.676673
Steady-State Error (%)	0.505677
Percent Overshoot (%)	0.000000

TABLE I: Performance metrics for the Closed-loop SEDM system (negative feedback).

Table I shows the performance metrics for the closed-loop SEDM system with negative feedback, providing key values like settling time and steady-state error.

Closed-loop PI-controlled system	
Metric	**Value**
ITAE	0.277496
Settling Time (s)	2.595372
Steady-State Error (%)	0.011846
Percent Overshoot (%)	30.237866

TABLE II: Performance metrics for the Closed-loop PI-controlled system .

Pre-filtered Closed-loop SEDM system (negative feedback)	
Metric	**Value**
ITAE	2.080703
Settling Time (s)	5.144465
Steady-State Error (%)	1.1401460
Percent Overshoot (%)	0.000000

TABLE III: Performance metrics for the Pre-filtered Closed-loop SEDM system (negative feedback).

Pre-filtered Closed-loop PI-controlled system	
Metric	**Value**
ITAE	1.026345
Settling Time (s)	3.947467
Steady-State Error (%)	0.242185
Percent Overshoot (%)	0.000000

TABLE IV: Performance metrics for the Pre-filtered Closed-loop PI-controlled system

Lastly, tables II I III IV display the performance metrics for the SEDM and closed-loop PI-controlled system with and without prefiltering. They demonstrate its ITAE and other relavent information.

III. Literature Review

As discussed in [2] , a new method of stability analysis for Dc motor drives is proposed, simulated, and tested . This method evaluates the stability of the open-loop system in response to gain and phase changes.From specifications of the transient response characteristics, steady state response characteristics, and gain and phase margin for the system, several algorithms were used to produce the optimal PID controller coefficients which meet these specifications for the DC motor drive The mathematical sets of acceptable gain and phase variations form a disk-shaped plot in the gain-phase plane, hence the name of the method. The open-loop response gain and phase variations among the PID control algorithms are plotted in the gain-phase plane, and evaluated to determine whether or not these control algorithms produce gain and phase variations that have membership within the acceptable disk-shaped gain and phase margin mathematical sets.Uncertainty on gain and phase variations for each controller algorithm's open-loop response of the DC motor drive's speed-control is also evaluated. A comparison of performance among the control algorithms for the DC motor drive's closed-loop transfer function speed control, torque response, and frequency response of the system is conducted. A comparison among the controller algorithms for both conventional stability margin analysis and the proposed disk-shaped stability margin analysis is also performed.The article concludes that the proposed method is an effective method for stability analysis of the DC motor drive system.

In [3] an alternative method for determining PI coefficients using a method called metrical optimum criteria aims to be an improvement over the Ziegler-Nichols method. In DC drives the purpose of the closed-loop control is to increase the steady state accuracy and to improve the speed response. This is to prevent a large current flowing through the armature and drive whenever the motor drive is started. In the topology used for their DC motor drive the PI feedback loop does not impact the operation of the drive, its only purpose is to return the drive back down to its rated armature current. Once at that setpoint, the PI control loop is deactivated automatically. The topology of this setup is a speed controller, PI current controller, and DC motor with a speed sensor in the feedback. The response of the controller is described in terms of its responsiveness to an error signal, the overshoot over the setpoint, and the amount of oscillations. A PI controller was used since the derivative action is more prone to measurement noise. Juxtaposed with a PI control where the main concern is that certain combinations may prevent it from reaching its setpoint. The Ziegler-Nichols method for tuning is popular and used very often in many control systems. It is described as the tangent line to the open-loop response curve. L is the distance from the origin to the intersection point of the tangent line to the x-axis. T is the distance between the tangent line's intersection with the x-axis to the tangent line's intersection with the setpoint. These are then put into equations to generate kp and Ki values. The symmetrical optimum criterion optimizes the frequency response of the system at low frequencies and aims to maximize the phase margin. This improves stability and disturbance rejection when compared to the Ziegler-Nichols method. The Kp and Ki values are determined from the closed-loop transfer function of the speed controller. The findings of this paper were that this method was an all-around improved method when comparing peak overshoot, peak time, rise time, and settling time when compared to the s-curve method.

As a means to reduce the variability of a system when receiving an input, a PID control algorithm was implemented based on an enhanced dung beetle optimizer and a back propagation neural network [4]. It goes into detail about using the natural behaviour of a dung beetle to avoid the issue of getting stuck in the local minima of a function. This is done by designing the search algorithm to prioritize the exploration of the function at the start of the algorithm. As the search progresses, the system starts to take into account the solutions found and starts to explore near the best solutions found. The value of this algorithm is that early on, local minima don't dictate the minimum value that will be found. Rather, it is stored as a point for local exploration among other possible locations. Using information from the data set created by the EDBO, a neural network is implemented, taking into account the error

signatl, system input, and the output of the system. Once taken into account, Kp, Ki, and Kd parameters are outputted from the system. When tested, it was found the combination of the EDBO with the neural network was better than any other neural network or optimization technique for finding the best system transient response. Due to the conclusions of this test, a similar experiment was done combining the testing methodology of the paper using a NARMA-L2 controller with the implementation of the neural network of this paper to conduct the experiment in this research paper.

While not looking at a PI controller, [5] uses the enhanced dung beetle algorithm in the turning of an active disturbance rejection controller (ADRC) for a permanent magnet synchronous motor (PMSM) speed control system. The algorithms that it was compared against are particle swarm optimization (PSO), gray wolf algorithm (GWO), sparrow algorithm (SSA), whale algorithm (WOA), and dung beetle algorithm (DBO) on a test of 23 benchmark functions of CEC2005. Similar to the PID control algorithm paper using EDBO the algorithm provided the best solution accuracy, convergence speed, and stability when compared to other algorithms. The paper concludes with its use case in other fields of engineering optimization, such as image segmentation, face recognition, data analysis, and more.

The article [6] employs the use of a grid search algorithm for hyperparameter tuning to improve sentiment analysis performance. It is described as an iterative process which goes through every possible combination of data. It is computationally intensive and can generate some unwanted data making discretion important. In this case, it was used to determine the best inputs for training an AI model.

This article [7] focuses on the use of a PI controller for the use of reducing the steady state error of a separately excited DC motor. In order to simulate the motor, a transfer function was used to express the characteristics of the SEDM. A PI controller is used to prevent output changes as long as an error is still found in the system. The value of the PI controller is that it removes these errors in the integration mode however, this can occasionally worsen the overall transient response of the system if not tuned correctly. The PI controller is composed of a Ki (integral gain) and Kp term (proportional gain). When KI is too small, a slow response can be observed; if it is too small, excess oscillations will be observed. This paper implemented the PI controller using two separate methods, one of which was via the Ziegler-Nicholas method and the second method was the NARMA-L2 controller in matlab. The NARMA-L2 controller is a neural network that allows for easy training of a system without needing a lot of computing resources. In the case of this neural network, the Input voltage, armature current, and the output of the system were used as input for the network to minimize oscillations and return the system to its default response. The paper concluded that the NARMA-L2 was a successful approach to tuning a neural network as a replacement for a PI controller. It didn't output Ki and Kp values and returned the system to its original state regardless of load or speed variations. However, it did not improve the transient response of the system when compared to its open loop configuration.

IV. Transfer Functions

SEDM Equation: [7]

$$T_1(s) = \frac{\frac{1}{0.005s+0.5} \cdot \frac{1}{1.3s+0.01}}{1 + \frac{1}{0.005s+0.5} \cdot \frac{1}{1.3s+0.01}} \tag{1}$$

$$T_1(s) = \frac{1}{0.0065s^2 + 0.65005s + 1.005} \tag{2}$$

General Closed Loop Equation: [4] [7]

$$T_2(s) = \frac{\frac{K_p + \frac{K_i}{s}}{0.0065s+0.65005s+1.005}}{1 + \frac{K_p + \frac{K_i}{s}}{0.0065s+0.65005s+1.005}} \tag{3}$$

$$T_2(s) = \frac{K_p + \frac{K_i}{s}}{0.0065s^2 + 0.65005s + 0.005 + K_p + \frac{K_i}{s}} \tag{4}$$

$$T_2(s) = \frac{K_p s + K_i}{0.0065s^3 + 0.65005s^2 + 0.005s + K_p s + K_i} \tag{5}$$

Addition of Pre-Filtering: [1]

$$P_f(s) = \frac{79}{s^2 + 79s + 79} \tag{6}$$

$$P_f(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{7}$$

$$P_f(s) \cdot T_1(s) = \frac{79}{s^2+79s+79} \cdot \frac{1}{0.0065s^2+0.65005s+1.005} \tag{8}$$

$$P_f(s) \cdot T_2(s) = \frac{79}{s^2+79s+79} \cdot \frac{K_p + \frac{K_i}{s}}{0.0065s^2+0.65005s+0.005+K_p+\frac{K_i}{s}} \tag{9}$$

The prefilter was designed in a way such that it would be an overdamped system to reduce any oscillations from suboptimal K_p and K_i values. Furthermore, the natural frequency was chosen so that rise time wouldn't be significantly slower than that of the SEDM [7].

V. Implementation of Algorithms for a Robust PID Controller

A. Enhanced Dung Beetle Optimization (EDBO)

The Enhanced Dung Beetle Optimizer (EDBO) [4] is an improved version of the standard Dung Beetle Optimizer (DBO). DBO is the foundation for searching through a search space for optimal results. However, it is susceptible to getting limited to specific local optima, preventing it from finding better solutions. The EDBO aims to circumvent this flaw, making it more reliable and improving its overall performance.

```python
class EDBO_PI:
    def __init__(
        self, obj_func, dim, bounds,
        pop_size=30, max_iter=100
    ):
        self.obj_func = obj_func  # Objective function
        self.dim = dim  # Number of parameters (Kp, Ki)
        self.bounds = np.array(bounds)
        self.pop_size = pop_size  # Number of dung beetles
        self.max_iter = max_iter  # Number of iterations

        # Initialize population
        self.population = np.random.uniform(
            self.bounds[:, 0],
            self.bounds[:, 1],
            (pop_size, dim)
        )

        self.best_solution = None
        self.best_fitness = float("inf")
        self.history = []
```

Fig. 4: Python Implementation of EDBO Initialization

In figure 4, bounds, the number of beetles, and iterations are defined. The beetles are data points that are randomly distributed across the bounds. They act as the initial values for the optimization function. In the case of this project, 30 points were used initially.

```python
h = np.random.uniform(0,1)
# Increase range for more movement
I = np.random.uniform(0.5, 1.5)

x_s = np.random.choice(
    self.population[np.argsort(self.fitness)[:5]]
)  # Pick from top 5

merit_position = self.population[i] + h * (
    x_s - I * self.population[i]
)

self.population[i] = np.clip(
    merit_position, self.bounds[:, 0], self.bounds[:, 1]
)
```

Fig. 5: Merit-oriented Mechanism

When looking for a local optimum, the algorithm in figure 5 will move in the direction of one of the top 5 solutions based on fitness. The variable h determines how large a step is taken; as h increases, the step becomes larger. On the other hand, I controls whether exploration or exploitation is prioritized. A smaller I encourages exploration, avoiding local minima. As iterations increase, I increases as well, shifting toward exploitation. This creates a balanced search that evolves overtime.

```python
r_min, r_max = 0.1, 1.0  # Ensuring a valid range
t_theta = np.tan(np.random.uniform(0, np.pi))
# Random angle

r = r_min + (r_max - r_min) * np.sin(
    np.pi * t / self.max_iter
)

prev_position = (
    self.population[i-1] if i > 0 else self.population[i]
)

self.population[i] = (
    r * self.population[i] +
    (1 - r) * t_theta * np.abs(
        self.population[i] - prev_position
    )
)

self.population[i] = np.clip(
    self.population[i], self.bounds[:, 0], self.bounds[:, 1]
)
```

Fig. 6: Sine Learning Factor for Balancing Exploration & Exploitation

The code in figure 6 is used to ensure a balance between exploration and exploitation. It ensures that early iterations are more explorative, whereas later iterations focus on fine-tuning local minima. Due to being a sinusoidal function limited in the range of 0.1 to 1, this function will increase until the argument in the sine function reaches $\pi/2$, then it will continue to decrease until almost becoming 0. This ensures that r oscillates in accordance to the behaviour of the loops as they progress, with an increasing r denoting exploration and a smaller r value denoting exploitation of local minima.

```python
# Dynamic Spiral Search for Fine-Tuning
q = np.exp(np.cos(np.pi * t / self.max_iter))
l = np.random.uniform(-1, 1)

spiral_move = q * np.cos(2 * np.pi * l) * np.random.uniform(-1, 1, self.dim)
self.population[i] += spiral_move
self.population[i] = np.clip(self.population[i], self.bounds[:, 0], self.bounds[:, 1])
```

Fig. 7: Dynamic Spiral Search for Fine-Tuning

Figure 7 ensures that the initial searching algorithm has a board range that decreases in size as the loops progress. At first q is large which encourages exploration as q shrinks exploration becomes exploitation.

```python
# Adaptive t-Distribution Disturbance
if np.random.rand() < 0.5:  # Increase probability of disturbance
    self.population[i] += np.random.standard_t(
        df=np.random.randint(3, 10), size=self.dim
    )
self.population[i] = np.clip(self.population[i], self.bounds[:, 0], self.bounds[:, 1])
```

Fig. 8: Adaptive t-Distribution Disturbance

Due to Figure 8 being a t distribution, when t is small, it replicates a Cauchy distribution, which encourages exploration. As the number of iterations progresses, it becomes Gaussian, which is better for the exploitation of local minima.

B. Grid Search Algorithm)

Another algorithm was designed to act as a comparison to to the EDBO, referred to as grid search [6] it applies a grid of two 2-dimensional inputs. In this case, it tests every combination of K_p and K_i for a given step size and returns their ITAE.

```
# Generate parameter grid
Kp_values = np.arange(bounds[0][0], bounds[0][1], step_size)
Ki_values = np.arange(bounds[1][0], bounds[1][1], step_size)
param_grid = [(Kp, Ki) for Kp in Kp_values for Ki in Ki_values]
```

Fig. 9: Parameter Grid Generation for Kp and Ki

VI. Training of Artificial Neural Networks (ANNs)

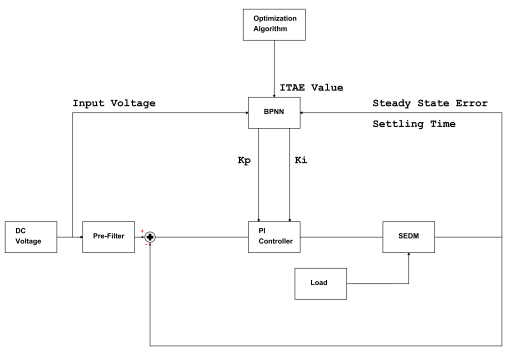

Fig. 10: Training of Neural Networks

The neural network was trained using Python tensor flow. Just as in the paper [4], both datasets were trained using a gradient descent algorithm via the AdamW gradient descent algorithm. For training, 3 input features were used (ITAE, settling time, and steady state error), and there was one hidden layer containing 4096 neurons and 2 neurons for output (K_p and K_i).

VII. Robust Control System Design using ANNs

Both ANNs were given the task of providing a K_p and K_i pair for an ITAE of 0.002, a settling time of 0.05s, and a steady state error of 0.0002. The EDBO and Grid search gave K_p and K_i of (21.68 and 36.6) and (27.33 and 47.8) respectively.

EDBO Closed-loop PI-controlled system	
Metric	**Value**
ITAE	0.001935
Settling Time (s)	0.071295
Steady-State Error (%)	0.000015
Percent Overshoot (%)	0.010893

TABLE V: Performance metrics for EDBO Closed-loop PI-controlled system

EDBO Pre-filtered Closed-loop PI-controlled system	
Metric	**Value**
ITAE	0.997629
Settling Time (s)	3.898687
Steady-State Error (%)	0.238147
Percent Overshoot (%)	0.000000

TABLE VI: Performance metrics for EDBO Pre-filtered Closed-loop PI-controlled system .

Grid Search Closed-loop PI-controlled system	
Metric	**Value**
ITAE	0.001896
Settling Time (s)	0.090056
Steady-State Error (%)	0.000012
Percent Overshoot (%)	2.963043

TABLE VII: Performance metrics for Grid Search Closed-loop PI-controlled system .

Grid Search Pre-filtered Closed-loop PI-controlled system	
Metric	**Value**
ITAE	0.990914
Settling Time (s)	3.891182
Steady-State Error (%)	0.236530
Percent Overshoot (%)	0.000000

TABLE VIII: Performance metrics for Grid Search Pre-filtered Closed-loop PI-controlled system .

VIII. Simulation Results

Using MATLAB simulink the EDNO [4], and grid search algorithm [6] were compared to the SEDM and PI controlled SEDM [7] in various simulation tasks with and without prefiltering [1].

A. Varying Voltage

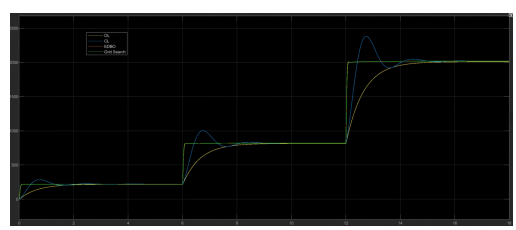

Fig. 11: Varying Voltage Without Prefilter

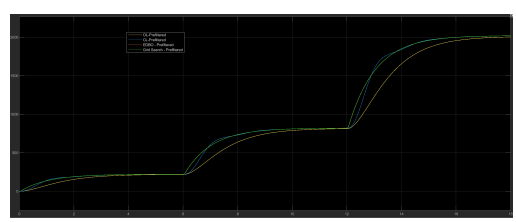

Fig. 12: Varying Voltage With Prefilter

B. Varying Load

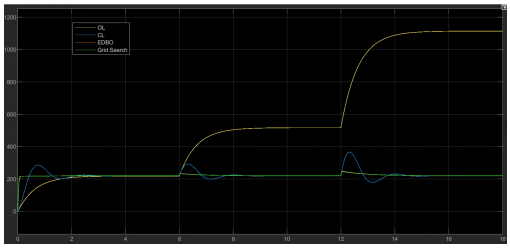

Fig. 13: Varying Load Without Prefilter

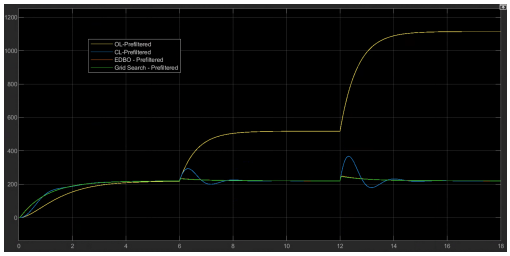

Fig. 14: Varying Load With Prefilter

C. Addition of Noise

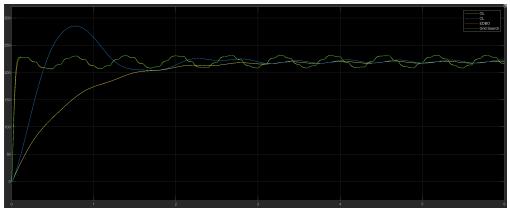

Fig. 15: Noise Without Prefilter

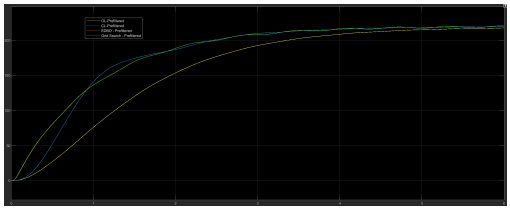

Fig. 16: Noise With Prefilter

IX. Discussion

A. Varying Voltage

In figure 11, we see the system multi-step response to input voltage variation for the Open-loop, closed- loop, EDBO, and Grid-search control systems. The EDBO and Grid-search control systems have the best response to input voltage variations, while the closed-loop control system has the worst. We see nearly identical responses for the EDBO and Grid-search control methods, which have very fast rise and settling times (almost immediate), negligible overshoot, and nearly zero steady state error. The open-loop response has no overshoot, settling time, or steady state error, but this comes at the cost of significantly longer rise time. The closed loop response has shorter rise time than the open-loop response but significantly longer rise time than both EDBO and Grid-search control systems. Additionally, the closed loop response has the worst overshoot, steady-state error, and settling time of all four control systems.In figure 12 , we again see the system multi-step response to input voltage variation for the four control systems, but this time we have included a pre-filter. Again, we see that the EDBO and Grid-search control systems have the best performance, but now we can see that closed-loop system has better response than the open-loop system. The EDBO and Grid-search control systems have increased rise and settling time compared to without the prefilter, but steady state error remains nearly non-existent as before. The closed-loop control system has no overshoot, steady state error remains the same as before, but the rise time and settling time has increased significantly with the addition of the prefilter. The open-loop control system still has no overshoot and steady state error remains as before, but the rise and settling times have increased with the addition of the prefilter.

B. Varying Load

In figure 13, we see the system multi-step response to load variation for the Open-loop, closed- loop, EDBO, and Grid-search control systems. The EDBO and Grid-search control systems have the best response to load variations, while the closed-loop control system has the worst. We see nearly identical responses for the EDBO and Grid-search control methods, which have very fast rise and settling times (almost immediate), minimal overshoot, and nearly zero steady state error.The open-loop response has no overshoot, settling time, or steady state error, but this comes at the cost of significantly longer rise time. As well, we see a staircase-like effect on the open-loop control system due to the lack of feedback to maintain the load setpoint. In this experiment, the load is applied every 6 seconds; without feedback, the load increases indefinitely as seen in the figure 13. The closed loop response has shorter rise time than the open-loop response but significantly longer rise time than both EDBO and Grid-search control systems. Additionally, the closed loop response has the worst overshoot, steady-state error, and settling time of all four control systems.In figure 14 , we again see the system multi-step response to load variation for the four control systems, but this time we have included a pre-filter.Again, we see that the EDBO and Grid-search control systems have the best performance, but now we can see that closed-loop system has better response than the open-loop system. The EDBO and Grid-search control systems have increased rise and settling time compared to without the prefilter, and there is some minimal steady state error .The closed-loop control system has similar overshoot as without the prefilter, and steady state error remains the same as before, but the rise time and settling time has increased significantly with the addition of the prefilter. The open-loop control system still has no overshoot and steady state error remains as before, but the rise and settling times

have increased with the addition of the prefilter. Overall, due to the prefilter not being a part of the feedback network the initial system response is the only difference between figures 13 and 14 as it has no impact on the system's ability to adjust to load.

C. Addition of Noise

In figure 15, we see the effects of noise on the system. Again EDBO and Grid-search control systems have nearly identical responses with the fastest rise and settling times, and nearly nonexistent overshoot, but the noise has introduced substantial steady state error. The open-loop system has the best steady state error and overshoot, but still the slowest rise and settling times.The closed loop system has the worst over shoot, has better rise and settling times than that of the open loop but worse than that of EDBO and Grid-search systems. The Closed loop system has better steady-state error than that of EDBO and Grid-search systems, but worse steady state error than that of the open-loop system.In figure 16 , we again see the effects of noise on the system but this time with the addition of the prefilter.The prefilter has substantially improved the steady state error for all four control systems but at the cost of longer rise and settling times. The EDBO and Grid-search control systems have nearly identical responses. They do have significantly worse rise and settling time compared to without the prefilter. The closed-loop system has slightly faster rise and settling time, and better steady state error than both of EDBO and Grid-search systems.The open-loop system has the best steady-state error but slowest rise and settling times. Thus the closed-loop system has the best performance out of the four control systems in response to noise.

The effects of the prefiltering are due to the system being overdamped. This is characterized by a slower response time as well as a reduction of osculations as seen in the graph.

X. Conclusion and next steps

The PI controlled feedback from article [7] provided overall worse characteristics than the EDBO [4] and gird search algorithms [6]. Just as in papers [4] and [5] the EDBO was found to provide the best characteristics compared to any other algorithm tested. The only concern with the algorithm is the difficulty it has with reducing noise. Both the grid search and EDBO were trained using the transfer function without the prefilter which is likely what contributed to its poor performance. Further testing of this system should include more varied datasets to determine both algorithms ability optimize K_p and K_i for prefiltering.

Additionally, while significantly better at finding the best ITAE values for a K_p and K_i pair. EDBO is significantly more computationally intensive to run for large data sets when compared to the grid search algorithm. However, it is unclear if it is due to poor coding implementations. It is important to note that overall, the neural network generated by the EDBO is more accurate over a larger range of values when compared to the grid search algorithm. A more thorough testing for this project would also include the use of multiple ITAE, steady state error and settling time to properly determine the better algorithm.

A final note, is that the data set for the gird search algorithm required 2 million data points to be somewhat useable. This is significantly more than the approximately 500 000 data points used to train the EDBO ANN. Further testing would also include variations in data set sizes to determine the better algorithm.

This paper succeeded in creating a robust PI-controlled system through the use of the EDPO, gridsearch and a neural network to generate K_p and K_i values. Those algorithms when compared to approaches like the s-curve method [7] are significantly better. However, when compared to another ANN and when introducing more varied systems (ie filtering) significantly more testing would be required for conclusive results as to what is the best method of creating robust control systems for DC motors.

REFERENCES

[1] R. H. Bishop, *Modern Control Systems Analysis and Design Using Matlab and Simulink*, Supplement ed. Boston, MA, USA: Addison-Wesley, 2007. ISBN: 978-0201498462.

[2] P. Gopi, "Novel approach to stability analysis of DC drive with parameter uncertainty and perturbation in feedback system," *Compel*, vol. 42, no. 6, pp. 1263–1284, Nov. 2023, doi: 10.1108/COMPEL-08-2022-0276.

[3] "Design of PI speed controller for 3-Ph converter fed DC motor drive using symmetrical optimization," *Journal of Applied Science and Engineering*, vol. 24, no. 6, pp. 837–844, Dec. 2021, doi: 10.6180/jase.202112_24(6).0003.

[4] W. Kong, H. Zhang, X. Yang, Z. Yao, R. Wang, W. Yang, and J. Zhang, "PID control algorithm based on multistrategy enhanced dung beetle optimizer and back propagation neural network for DC motor control," *Scientific Reports*, vol. 14, no. 1, pp. 28276–28276, Nov. 2024, Article no. 28276, doi: https://doi.org/10.1038/s41598-024-79653-z.

[5] Z. Lu and G. Guifang, "Active disturbance rejection control of permanent magnet synchronous motor based on improved dung beetle algorithm," in *Proc. 2024 3rd Int. Conf. on Cyber Security, Artificial Intelligence and Digital Economy (CSAIDE)*, 2024, pp. 629–633, doi: 10.1145/3672919.3673029.

[6] I. Priyadarshini and C. Cotton, "A novel LSTM–CNN–grid search-based deep neural network for sentiment analysis," *The Journal of Supercomputing*, vol. 77, no. 12, pp. 13911–13932, Dec. 2021, doi: 10.1007/s11227-021-03838-w.

[7] B. Ullah, S. Hussain, M. Yousuf, F. Khan, S. Khalid, S. Akbar, and A. Muhammad, "Speed control of separately excited DC motor using NARMA-L2 controller," in *Proc. 2022 Int. Conf. on Technology and Policy in Energy and Electric Power (ICT-PEP)*, 2022, pp. 43–47, doi: 10.1109/ICT-PEP57242.2022.9988795.